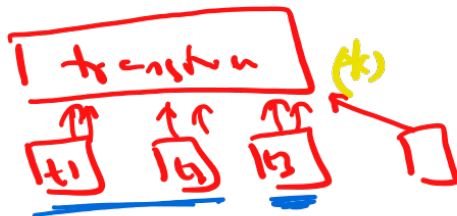


HOW KV-CACHE AND ATTENTION SCALE WITH SEQUENCE LENGTH (n)

MLP or ffw.

Independently
for each token.



$$O(n^2)$$

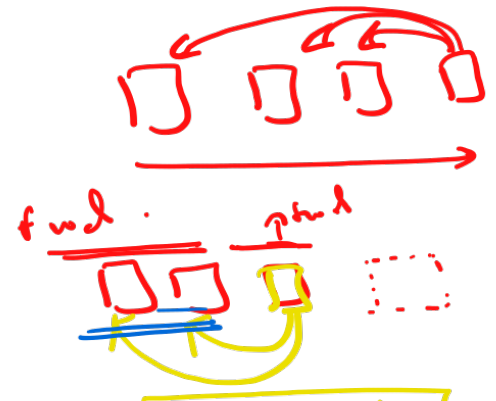


$$O(n)$$

Without
kv
cache

with kv
cache .

Attention .

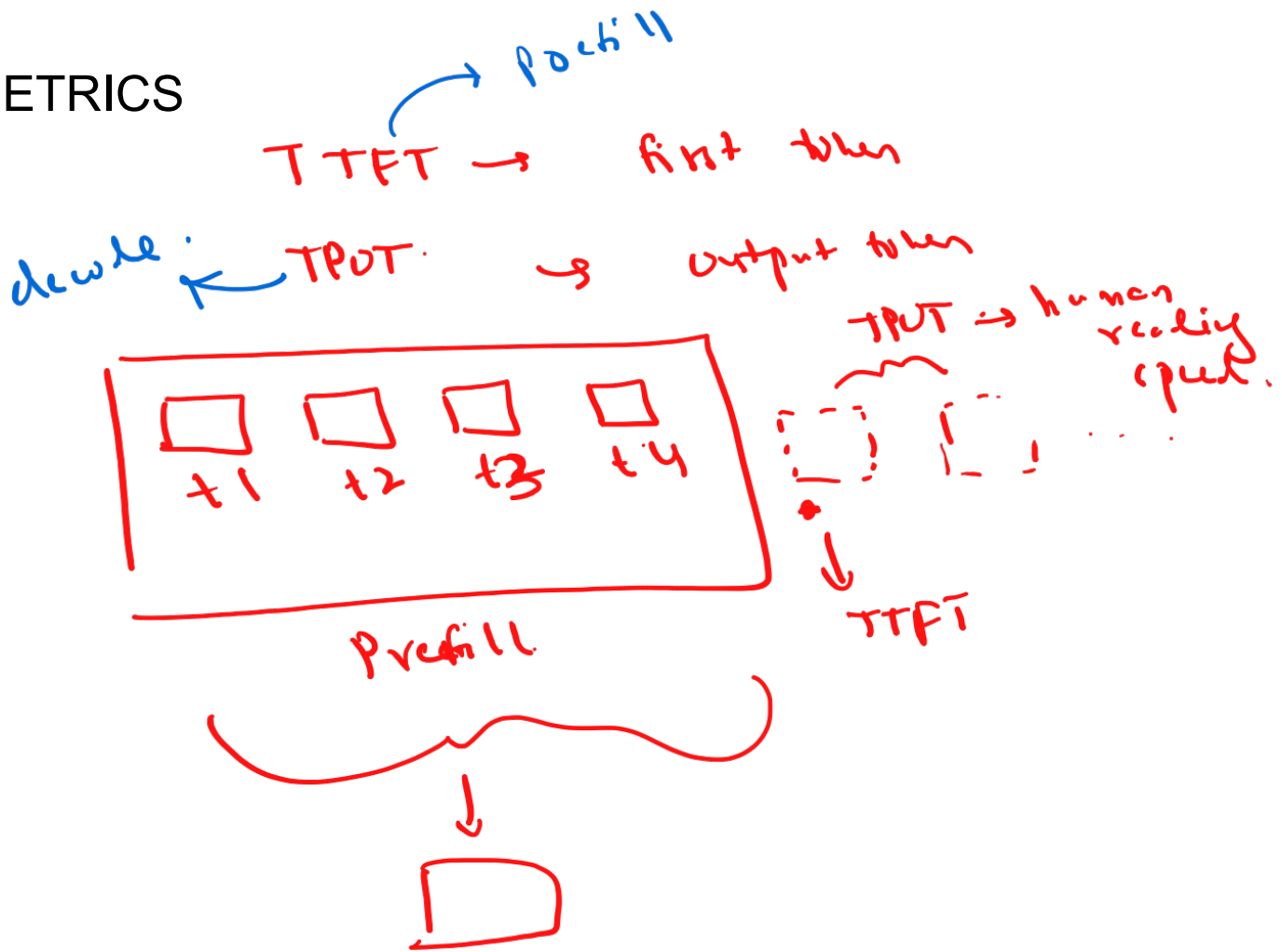


$$O(n^3)$$

Dependent on previous
tokens.

$$O(n^2)$$

METRICS



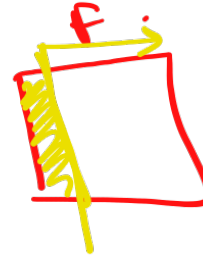
BOTTLENECKS

LINEAR

Row Line



(a)



all in

$D + D - 1$
multiplication
(20-1)

f wps.

$2BDF$

Communication = $\frac{b f l b.}{2 \text{ bytes.}}$

Read -

$2BD + 2DF + 2BF$

Write.

$$T_{\text{math}} = \frac{2BDF}{C}$$

C

accelerator flops

$$T_{\text{comm}} = \frac{2BD + 2DF + 2BF}{\text{bandwidth (B)}}$$

$$C = 1.97e^{14}$$

$$BW = 7.2e^{11}$$

TPU v5e.

T_{math}

$$\frac{2BDF}{1.97e^{14}}$$

T_{writing}

$$\frac{2BD + 2DF + 2BF}{7.2e^{11}}$$

$D, F \gg B.$

$$\frac{2BDF}{\text{inference (B)}} \gg \frac{1.97e^{14}}{7.2e^{11}} \sim 270$$

$$\frac{2 B D F}{2 D F} \geq 240$$

Critical
batch
size

$$B \geq 240$$

$$\frac{2 B D F}{2 D F}$$

$\approx$

$$B \geq \frac{C}{w}$$

bf 16

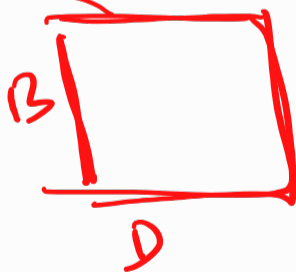
Flops (C) $\uparrow$ $\rightarrow$ Batch size
to be
compute and $\uparrow$

$$B \geq \frac{C_{int}}{w}$$

$B \uparrow$

$w \uparrow$ $\rightarrow$ Batch size to
be compute and $\downarrow$

activations



@



Bits per param /
bits per activation

$$D, w \gg 1$$

$$T_{matr} = 2 B D F / C$$

$$\alpha = 1 (\text{int8})$$

$$\alpha = 2 (\text{bf16})$$

$$T_{units} = \frac{\alpha \cdot D F}{w}$$

b_w = bits per
parameter

$$b_w = 16 (\text{bf16})$$

$$\alpha = 2$$

$$b_w = 2 (\text{int8})$$

$$\alpha = 1$$

$$R_{outline} = \frac{2 B D F}{B C}$$

$$\frac{\alpha D F}{w}$$

$$b_w = \frac{d}{2}$$

$$T_{wms} = \frac{\frac{bw}{\gamma} DF}{W.}$$

$$bw = 16 \text{ (bf)} \\ \bullet \text{ } 2DF \\ bw = 3 \text{ (int)} \\ DF.$$

$$T_{mch} = \frac{2BDF}{C}$$

$$\text{int} \rightarrow \boxed{C_{int} = 2C_{bf}}$$

$b_a = \text{bits per action}$

$$= \frac{2BDF}{C_{bf} \cdot \frac{16}{b_a}}$$

$$b_a = 16 \text{ (bf)}$$

$$b_a = 3 \text{ (int)}$$

$$C_{bf} \times \frac{16}{3} = \boxed{2C_{bf}}$$

$$\frac{2BDF}{C_{bf}}$$

$$P_{wflr} = \frac{\frac{2BDF}{C_{bf} \cdot \frac{16}{b_a}}}{\frac{\frac{bw}{\gamma} DF}{W.}} \geq 1$$

$w \sim \text{put out.}$

$$\boxed{B_{crit} \geq \frac{C_{bf} \cdot \frac{bw}{b_a}}{W.}}$$

β

ATTENTION

Scale Your Model

Previous Part Next Part

but do our computation in bf16, the critical batch size drops to 120. With int8 activations and int8 params, it jumps back up to 240 since the TPUv5e can supply 400 TOPs/s of int8 x int8.

What about attention?

Things get more complicated when we look at the dot-product attention operation, especially since we have to account for KV caches. Let's look at just one attention head with pure multi-headed attention. In a single Flash Attention fusion, we³:

1. Read the Q activations of shape $\text{bf16}[B, T, D]$ from HBM. [B, T, D]
2. Read the KV cache, which is a pair of $\text{bf16}[B, S, D]$ tensors from HBM. 2BTD
3. Perform $2BSTD$ FLOPs in the QK matmul. With Flash Attention, we don't need to write the $\text{bf16}[B, S, T]$ attention matrix back into HBM. 2BSD x 2 = 4BSD
4. Perform $2BSTD$ in the attention AV matmul. (Flops) $B \times S \times S \times D$
5. Write the resulting $\text{bf16}[B, T, D]$ tensor back into HBM. huge write!!

Galaxy Tab S7 FE All together, we get:

$$\text{Routine} = \frac{2BSTD + 2BSTD}{2BTD + 4BSD + 2BTD}$$

→ QK^T → AV

Q (read) K & V (read) write back

$\text{Sum}(QK^T) @ V$

$$AI = \frac{4(3ST)}{2(3TD) + 4(3SD)} = \frac{ST}{S+T}$$

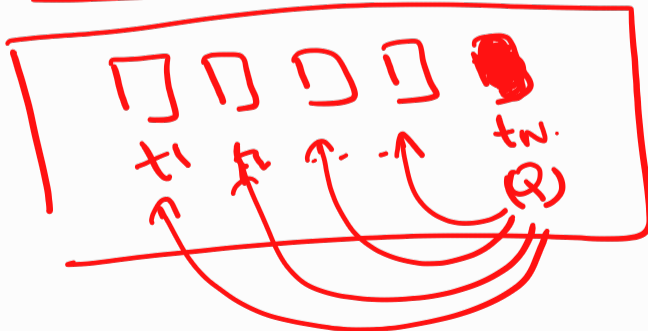
$S \uparrow$ or $T \uparrow$
compute and regime.
Pre fill

$$\frac{ST}{S+T}$$

Devide.

memory word
power AI.

$$\boxed{S=T} \quad \frac{S^2}{S+T} = \left(\frac{S}{2}\right) \text{ or } \left(\frac{T}{2}\right)$$



$$F=1$$

$$\frac{S}{S+1} \approx 1$$

Constant

$$\boxed{XXX}$$

$S/2$ or $T/2 \rightarrow$ scales with regime

THEORETICAL ESTIMATES FOR LLM LATENCY AND THROUGHPUT

memory

B ↓ → memory bound.

B. C_{kv} → total kv cache

+

P. → parameters size.

↓
You need to load kv cache to do ps.

$$\text{theoretical min step time } (T_{\text{step}}) = \frac{B C_{kv} + P}{\text{Bandwidth}}$$

$$\text{throughput (max tokens)} = \frac{B}{T_{\text{step}}}$$

$$(\text{max tokens}) = \frac{B \times \text{Bandwidth}}{B \cdot C_{kv} + P}$$

Total time step =

$T_{\text{attention}} + T_{\text{MLP}}$

↓

B W → memory bound.

↓

compute (ops) memory bound

W = total memory bandwidth

||

T_{wmmv}

$$= \frac{B \cdot C_{kv}}{W}$$

$$+ \left(\frac{T_{\text{math, MLP}}}{\text{max}} \right) T_{\text{wmmv}}$$

total
time step. =

$P \rightarrow$ Parameters
size -

$C \rightarrow$ FWPs

$$\frac{B \cdot C_{kv}}{W} + \overset{\text{fwd}}{\text{rev}} \left(\frac{2 \times B \times P}{C} \right), \frac{P}{W}$$

$\underbrace{\hspace{10em}}_{T_{\text{wmm}, \text{fwd}}}$
 $\underbrace{\hspace{10em}}_{T_{\text{mat}, \text{rev}}}$
 $\underbrace{\hspace{10em}}_{T_{\text{wmm}, \text{rev}}}$

$\text{fwd} \quad (\text{B} \rightarrow \text{P})$

MEMORY

$\frac{B}{T_{step}}$

$B \cdot C_{kv} + P$

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	6.7	53.6	107.2	214.4	428.8	1608
Total Memory (GiB)	32.7	79.6	133.2	240.4	454.8	1634
Theoretical Step Time (ms)	4.98	12.13	20.30	36.65	69.33	249.09
Theoretical Throughput (tokens/s)	200.61	659.30	787.99	873.21	923.13	963.53

8x TPU v5es gives us 128GiB of HBM, 6.5TiB/s of HBM bandwidth (0.82TiB/s each) and 1600TF/s of compute.

Memory kv cache $\rightarrow g_{kv-dim}$ $\rightarrow$ system.

$$2 \times H \times K \times L \times S$$

$(K+V)$ #heads #layers

$$2 \times 128 \times 40 \times 40 \times 8 \times 92 \approx 6.7 \text{ GiB}$$

40 heads
 $\hookrightarrow 1:5$ (8 heads)

$6.7/5$

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	1.34	10.72	21.44	42.88	85.76	321.6
Total Memory (GiB)	27.34	36.72	47.44	68.88	111.76	347.6
Theoretical Step Time (ms)	4.17	5.60	7.23	10.50	17.04	52.99
Theoretical Throughput (tokens/s)	239.94	1,429.19	2,212.48	3,047.62	3,756.62	4,529.34

$B \cdot C_{kv} + P$
 $\rightarrow 26 \text{ GiB}$

13B
 $\downarrow$
 6F16
 2x13 GB
 26GB
 $+ 6.7$
 $\hline$
 32.7

$\uparrow \uparrow$
 $\frac{1}{4.98}$

(1.5-2)

Diagram illustrating the input tensors for the model:

- A 3D tensor A (input sequence) with dimensions: seq len , hidden dim , and batch size .
- A 2D tensor W (weight matrix) with dimensions: F (number of features) and D (number of hidden units).

$y \approx \begin{cases} 16324 \\ 1200 \end{cases}$
 $\approx y$

- high & attribute

ing, we can overlap the communication and computation (using collective matmul for
gator and ring attention respectively).

$$T_{\text{matm}} = \frac{2BSDF}{Y \cdot C}$$

$$T_{\text{comm}} = \frac{2BSD}{Wici \cdot (1)} \quad \text{flops.}$$

$$\frac{T_{\text{matm}}}{T_{\text{comm}}} \geq 1 \Rightarrow \boxed{Y \leq F / (E/Wici)}$$

ICI $\rightarrow$ move / load
a character

$$Y \leq \frac{F}{C/wici}$$

$$\leq \frac{F}{\frac{1.97e^{14}}{9e^{10}}}$$

$$Y \leq F/w_{00}$$

tpuvse.

DECODE

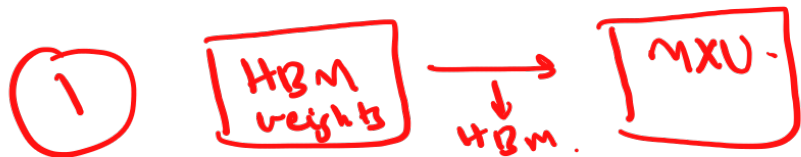
Generation

Generation is a more complicated beast than prefill. For one thing, it is harder to get a large batch size because we need to batch many requests together. Latency targets are lower. Together, these mean we are typically more memory-bound and more sensitive to communication overhead, which restrict our sharding strategies:

1. **FSDP is impossible**: since we are memory-bound in loading our parameters and KV caches from HBM to the MXU, we do not want to move them via ICI which is orders of magnitudes slower than HBM. *We want to move activations rather than weights.* This means methods similar to FSDP are usually completely unviable for generation.⁵
2. **There is no reason to do data parallelism**: pure data parallelism is unhelpful because it replicates our parameters and doesn't help us load parameters faster. You're better off spinning up multiple copies of the model instead.⁶
3. **No sequence = no sequence sharding**. Good luck sequence sharding.

memory bound

Takeaway: our only options during generation are variants of model parallelism. We aim to move activations instead of KV caches or parameters, which are larger. When our batch size is large, we do model parallelism up to the FLOPs-ICI bound (F/α). When our batch size is smaller, we can improve latency by model sharding more (at a modest throughput cost). When we want to model shard more ways than we have KV heads, we can shard our KVs along the batch dimension as well.



SOURCES

loading activations.

$$(1) \quad \text{bfc}[0, F] = \frac{2DF \xrightarrow{\text{weight matrix}}}{Y \cdot W_{\text{hbm}}}$$

$$(2) \quad \left(\frac{2BD}{W_{\text{ici}}} \right) \quad \begin{matrix} (k=1) \\ (\text{ICT}) \end{matrix}$$

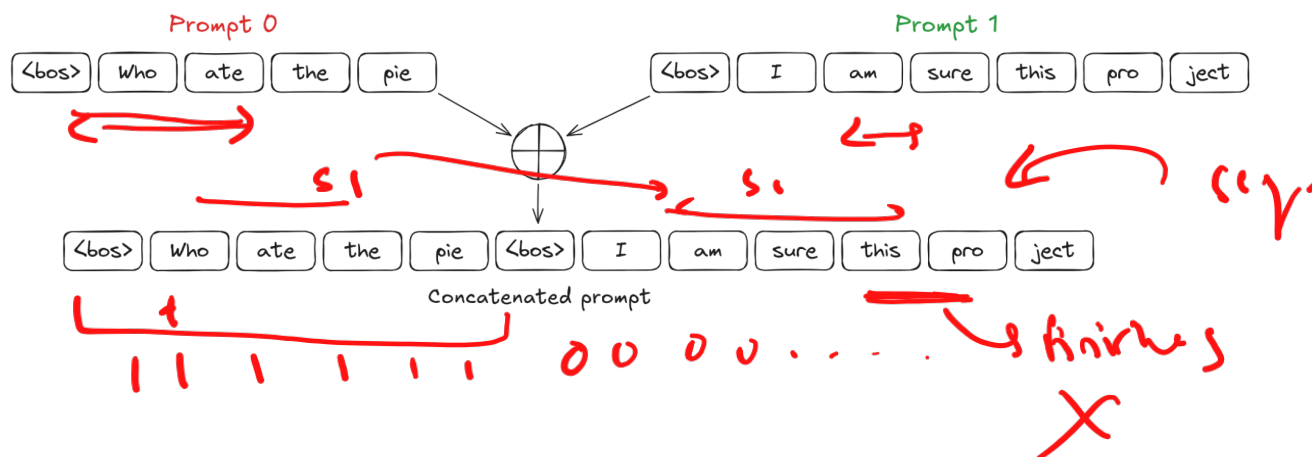
$$\frac{2DF}{Y \cdot W_{\text{hbm}}} \geq \frac{2DF}{W_{\text{ici}}}$$

$$W_{\text{hbm}} \approx 2x W_{\text{ici}}$$

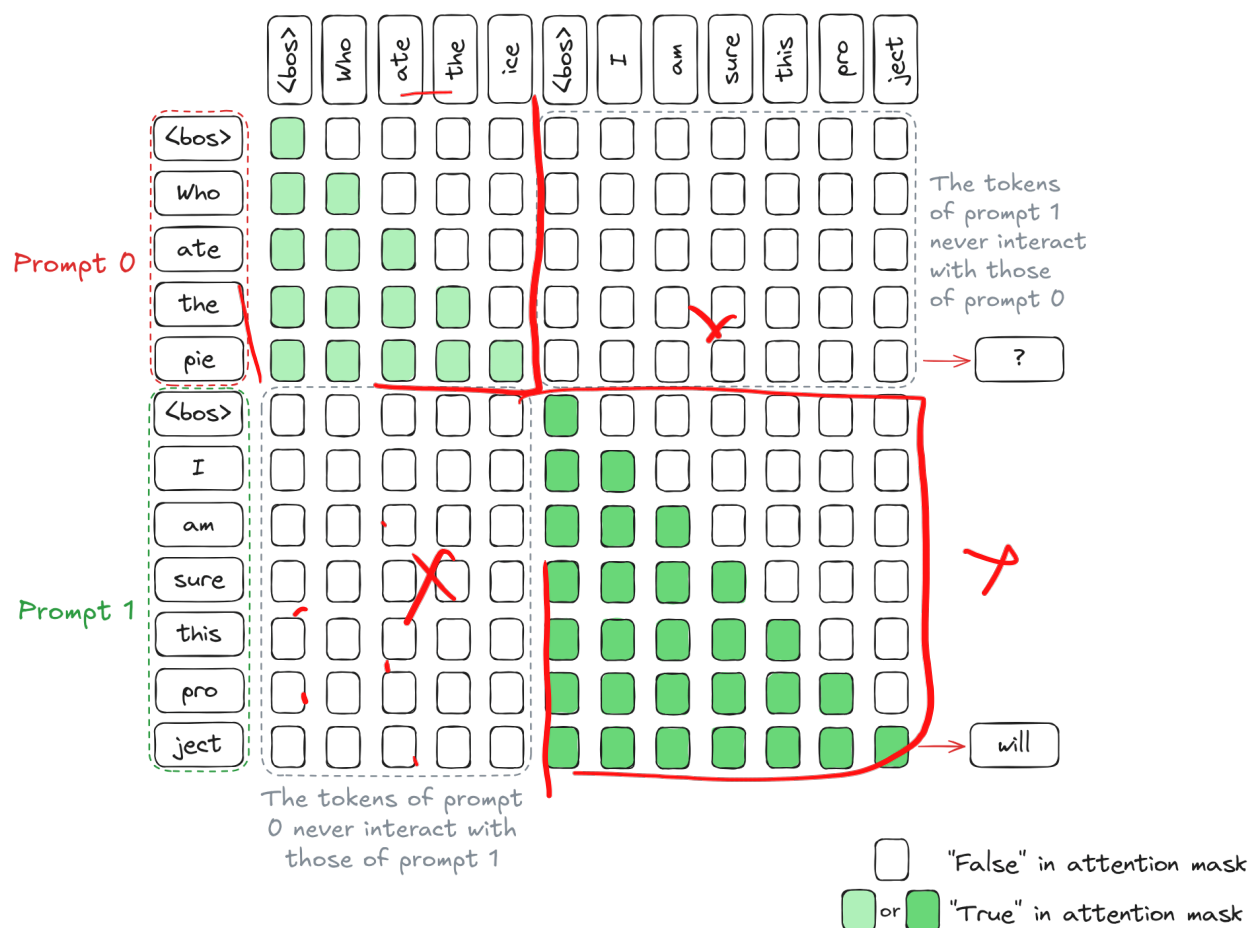
$$Y \leq \frac{F W_{\text{ici}}}{3 W_{\text{hbm}}}$$

$$Y \leq \frac{F}{3 \cdot 3}$$

CONTINUOUS BATCHING



Ragged batching



SHARDING THE KV CACHE (NEXT VIDEO)